PYTHON INFRAESTRUCTURA PKI

Asegúrese de instalar la biblioteca necesaria antes de ejecutar el script:

- pip install cryptography

```
PS C:\Users\2-DAW> pip install cryptography
Defaulting to user installation because normal site-packages is not writeable
Collecting cryptography
  Downloading cryptography-46.0.3-cp311-abi3-win_amd64.whl.metadata (5.7 kB)
Requirement already satisfied: cffi>=2.0.0 in c:\users\2-daw\appdata\local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\
site-packages (from cryptography) (2.0.0)
Requirement already satisfied: pycparser in c:\users\2-daw\appdata\local\packages\pythonsoftwarefoundation.python.3.13_qbz5n2kfra8p0\localcache\local-packages\python313\si
te-packages (from cffi>=2.0.0->cryptography) (2.23)
Downloading cryptography-46.0.3-cp311-abi3-win_amd64.whl (3.5 MB)
   ---------------------------------------- 3.5/3.5 MB 40.9 MB/s  0:00:00
Installing collected packages: cryptography
Successfully installed cryptography-46.0.3

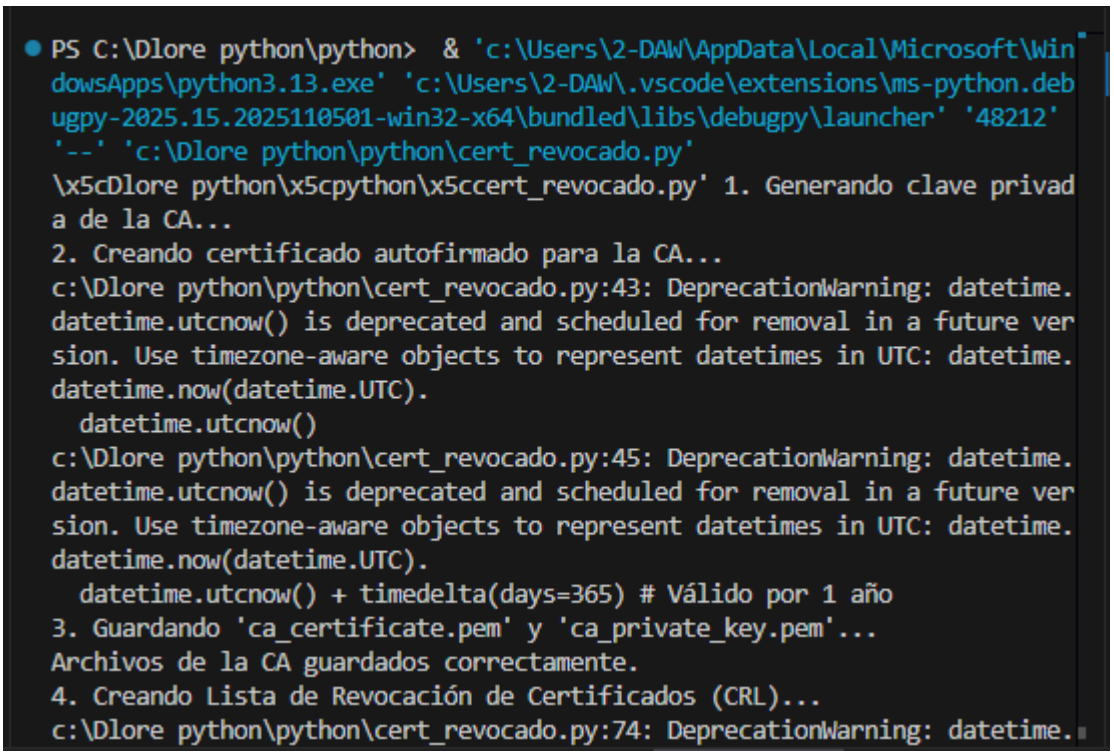
[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: C:\Users\2-DAW\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.13_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip
PS C:\Users\2-DAW>
PS C:\Users\2-DAW>
```

C: > Dlore python > python > cert_revocado.py > ...

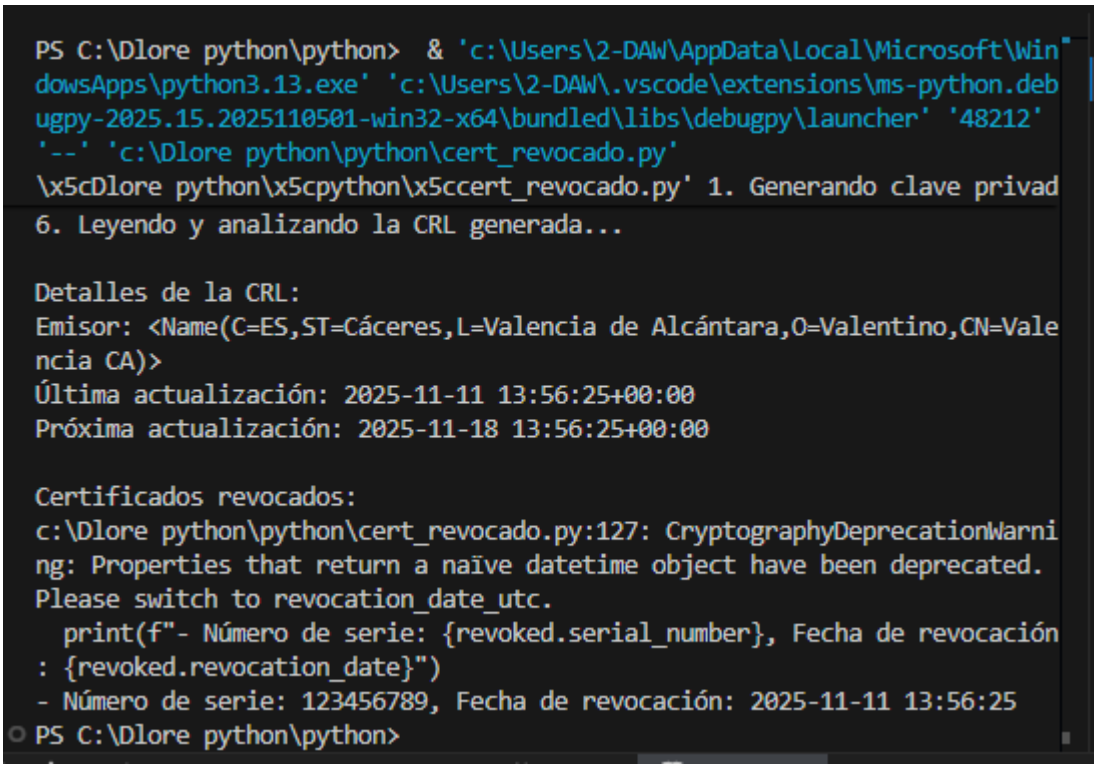
```python
from cryptography import x509
from cryptography.x509.oid import NameOID
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives.serialization import (
    Encoding,
    PrivateFormat,
    NoEncryption
)
from datetime import datetime, timedelta
import os # Importamos os para verificar la existencia de

# --- Función Principal ---
def create_ca_and_crl():
    # 1. Generar una clave privada para la CA
    print("1. Generando clave privada de la CA...")
    ca_private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )

    # 2. Crear un certificado autofirmado para la CA
    print("2. Creando certificado autofirmado para la CA..
```



```
PS C:\Dlore python\python>  & 'c:\Users\2-DAW\AppData\Local\Microsoft\Win
dowsApps\python3.13.exe' 'c:\Users\2-DAW\.vscode\extensions\ms-python.deb
ugpy-2025.15.2025110501-win32-x64\bundled\libs\debugpy\launcher' '48212'
'--' 'c:\Dlore python\python\cert_revocado.py'
\x5cDlore python\x5cpython\x5ccert_revocado.py' 1. Generando clave privad
a de la CA...
2. Creando certificado autofirmado para la CA...
c:\Dlore python\python\cert_revocado.py:43: DeprecationWarning: datetime.
datetime.utcnow() is deprecated and scheduled for removal in a future ver
sion. Use timezone-aware objects to represent datetimes in UTC: datetime.
datetime.now(datetime.UTC).
  datetime.utcnow()
c:\Dlore python\python\cert_revocado.py:45: DeprecationWarning: datetime.
datetime.utcnow() is deprecated and scheduled for removal in a future ver
sion. Use timezone-aware objects to represent datetimes in UTC: datetime.
datetime.now(datetime.UTC).
  datetime.utcnow() + timedelta(days=365) # Válido por 1 año
3. Guardando 'ca_certificate.pem' y 'ca_private_key.pem'...
Archivos de la CA guardados correctamente.
4. Creando Lista de Revocación de Certificados (CRL)...
c:\Dlore python\python\cert_revocado.py:74: DeprecationWarning: datetime.
```



```
PS C:\Dlore python\python>  & 'c:\Users\2-DAW\AppData\Local\Microsoft\Win
dowsApps\python3.13.exe' 'c:\Users\2-DAW\.vscode\extensions\ms-python.deb
ugpy-2025.15.2025110501-win32-x64\bundled\libs\debugpy\launcher' '48212'
'--' 'c:\Dlore python\python\cert_revocado.py'
\x5cDlore python\x5cpython\x5ccert_revocado.py' 1. Generando clave privad
6. Leyendo y analizando la CRL generada...

Detalles de la CRL:
Emisor: <Name(C=ES,ST=Cáceres,L=Valencia de Alcántara,O=Valentino,CN=Vale
ncia CA)>
Última actualización: 2025-11-11 13:56:25+00:00
Próxima actualización: 2025-11-18 13:56:25+00:00

Certificados revocados:
c:\Dlore python\python\cert_revocado.py:127: CryptographyDeprecationWarni
ng: Properties that return a naïve datetime object have been deprecated.
Please switch to revocation_date_utc.
  print(f"- Número de serie: {revoked.serial_number}, Fecha de revocación
: {revoked.revocation_date}")
- Número de serie: 123456789, Fecha de revocación: 2025-11-11 13:56:25
PS C:\Dlore python\python>
```

Estructura de los certificados de atributos

attribute_certificate_model.py

from cryptography import x509
from cryptography.hazmat.primitives import hashes
from cryptography.x509.oid import NameOID
from datetime import datetime, timedelta

```
import uuid

# --- 1. Definición Conceptual de las Clases ---
# En un sistema real, no defines las clases tú mismo.
# Usarías las clases predefinidas de la librería,
# pero esto ayuda a entender la estructura.

class AttributeCertificateInfo:
    """Simula la estructura AttributeCertificateInfo."""
    def __init__(self, version, holder_name, issuer_name, serial):
        self.version = version
        self.holder = holder_name
        self.issuer = issuer_name
        self.serialNumber = serial
        self.attrCertValidityPeriod = None # Lo estableceremos después
        self.attributes = []
        self.issuerUniqueID = None # Opcional
        self.extensions = [] # Opcional

    def add_attribute(self, oid_str, value):
        """Agrega un atributo, por ejemplo, un rol."""
        # En la vida real, los atributos son complejos y se representan como X.509 Extensions
o Attributes
        self.attributes.append({
            "oid": oid_str,
            "value": value
        })

    def set_validity(self, not_before, not_after):
        self.attrCertValidityPeriod = {
            "not_before": not_before,
            "not_after": not_after
        }

    def __str__(self):
        return f"""
        --- AttributeCertificateInfo ---
        Versión: {self.version}
        Titular (Holder): {self.holder}
        Emisor (Issuer): {self.issuer}
        No. Serie: {self.serialNumber}
        Validez: {self.attrCertValidityPeriod['not_before'].strftime('%Y-%m-%d')} a
{self.attrCertValidityPeriod['not_after'].strftime('%Y-%m-%d')}
        Atributos: {self.attributes}
        """

class AttributeCertificate:
    """Simula el AttributeCertificate ::= SIGNED { AttributeCertificateInfo }."""
```

```
    def __init__(self, info_obj):
        self.info = info_obj
        self.signature_algorithm = "SHA256withRSA" # Algoritmo de firma
        self.signature_value = "SIMULATED_SIGNATURE_DATA" # El resultado de la firma

    def __str__(self):
        return f"AttributeCertificate (Firmado con {self.signature_algorithm})\n{str(self.info)}"

# --- 2. Creación del Objeto ---

# 1. Definir los nombres para el Titular y el Emisor
holder_cn = "Usuario Autorizado"
issuer_cn = "Autoridad de Atributos del Departamento de TI"

# 2. Crear la información base
info = AttributeCertificateInfo(
    version=2, # v2 es el estándar
    holder_name=holder_cn,
    issuer_name=issuer_cn,
    serial=int(uuid.uuid4().int % (10**20)) # Número de serie grande simulado
)

# 3. Establecer el periodo de validez
now = datetime.utcnow()
info.set_validity(
    not_before=now,
    not_after=now + timedelta(days=365)
)

# 4. Agregar atributos (El Rol es un atributo clave en estos certificados)
# Usando OID (Object Identifier) simulado
info.add_attribute(
    oid_str="1.3.6.1.4.1.311.20.2", # OID común para "Role"
    value="Administrador de Sistemas"
)

# 5. Finalmente, crear el certificado completo con la firma (simulada)
attr_cert = AttributeCertificate(info)


# --- 3. Ejecución y Visualización en VS Code ---
print("--- Modelo Conceptual del Certificado de Atributo ---")
print(attr_cert)

# Para un caso de uso real, usarías 'cryptography' para leer o generar certificados X.509
print("\n--- Nota: Uso de Librería 'cryptography' para Certificados Reales ---")

# Ejemplo de cómo se vería una representación de Nombre del Sujeto real
```

```
subject_name_real = x509.Name([
    x509.NameAttribute(NameOID.COUNTRY_NAME, "ES"),
    x509.NameAttribute(NameOID.ORGANIZATION_NAME, "Mi Empresa"),
    x509.NameAttribute(NameOID.COMMON_NAME, holder_cn),
])

print(f"Representación real del nombre del titular (X.509): {subject_name_real}")
```

C: > Dlore python > python > cert_atributo.py > ...

```python
# attribute_certificate_model.py

from cryptography import x509
from cryptography.hazmat.primitives import hashes
from cryptography.x509.oid import NameOID
from datetime import datetime, timedelta
import uuid

# --- 1. Definición Conceptual de las Clases ---
# En un sistema real, no defines las clases tú mismo.
# Usarías las clases predefinidas de la librería,
# pero esto ayuda a entender la estructura.

class AttributeCertificateInfo:
    """Simula la estructura AttributeCertificateInfo."""
    def __init__(self, version, holder_name, issuer_name,
        self.version = version
        self.holder = holder_name
        self.issuer = issuer_name
        self.serialNumber = serial
        self.attrCertValidityPeriod = None # Lo establecer
        self.attributes = []
        self.issuerUniqueID = None # Opcional
        self.extensions = [] # Opcional
```

```
[Running] python -u "c:\Dlore python\python\cert_atributo.py"
c:\Dlore python\python\cert_atributo.py:76: DeprecationWarning:
datetime.datetime.utcnow() is deprecated and scheduled for removal in a
future version. Use timezone-aware objects to represent datetimes in
UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.utcnow()
--- Modelo Conceptual del Certificado de Atributo ---
AttributeCertificate (Firmado con SHA256withRSA)

        --- AttributeCertificateInfo ---
        Versi◆n: 2
        Titular (Holder): Usuario Autorizado
        Emisor (Issuer): Autoridad de Atributos del Departamento de TI
        No. Serie: 38946506023568043694
        Validez: 2025-11-12 a 2026-11-12
        Atributos: [{'oid': '1.3.6.1.4.1.311.20.2', 'value':
        'Administrador de Sistemas'}]
```

```
--- Modelo Conceptual del Certificado de Atributo ---
AttributeCertificate (Firmado con SHA256withRSA)

        --- AttributeCertificateInfo ---
        Versi◆n: 2
        Titular (Holder): Usuario Autorizado
        Emisor (Issuer): Autoridad de Atributos del Departamento de TI
        No. Serie: 38946506023568043694
        Validez: 2025-11-12 a 2026-11-12
        Atributos: [{'oid': '1.3.6.1.4.1.311.20.2', 'value':
        'Administrador de Sistemas'}]


--- Nota: Uso de Librer◆a 'cryptography' para Certificados Reales ---
Representaci◆n real del nombre del titular (X.509): <Name(C=ES,O=Mi
Empresa,CN=Usuario Autorizado)>
```

Código de simulación de atributos

C: > Dlore python > python > simulacion_atributos.py > ...

```
123    clave_ca = generar_clave_privada()
124    clave_aa = generar_clave_privada()
125
126    # 2. Creación del Certificado Digital (CA)
127    certificado_digital = crear_certificado_digital(clave_ca,
128
129    # 3. Definición de atributos y creación del Certificado de
130    atributos = "Rol: Administrador; Permiso: Acceso Completo"
131    certificado_atributo = crear_certificado_atributo(clave_aa
132
133    # 4. Exportación de los certificados a archivos
134    exportar_certificado(certificado_digital, "certificado_dig
135    exportar_certificado(certificado_atributo, "certificado_at
136
137    # 5. Mostrar resultados en consola
138    print("--- SIMULACIÓN DE PKI (CA + AA) ---")
139    print("\nCertificado Digital (CA):")
140    # Al imprimir, se muestra una representación del objeto (r
141    print(certificado_digital)
142
143    print("\nCertificado de Atributo (AA):")
144    print(certificado_atributo)
145
146    print("\n¡Exportación completada!")
147    print("Se han generado los archivos 'certificado_digital.p
```

```
c:\Dlore python\python\simulacion_atributos.py:53: DeprecationWarning:
datetime.datetime.utcnow() is deprecated and scheduled for removal in a
future version. Use timezone-aware objects to represent datetimes in
UTC: datetime.datetime.now(datetime.UTC).
  datetime.utcnow()
c:\Dlore python\python\simulacion_atributos.py:55: DeprecationWarning:
datetime.datetime.utcnow() is deprecated and scheduled for removal in a
future version. Use timezone-aware objects to represent datetimes in
UTC: datetime.datetime.now(datetime.UTC).
  datetime.utcnow() + timedelta(days=365) # V□lido por 1 a□o
c:\Dlore python\python\simulacion_atributos.py:93: DeprecationWarning:
datetime.datetime.utcnow() is deprecated and scheduled for removal in a
future version. Use timezone-aware objects to represent datetimes in
UTC: datetime.datetime.now(datetime.UTC).
  datetime.utcnow()
c:\Dlore python\python\simulacion_atributos.py:95: DeprecationWarning:
datetime.datetime.utcnow() is deprecated and scheduled for removal in a
```

```
Traceback (most recent call last):
  File "c:\Dlore python\python\simulacion_atributos.py", line 131, in
  <module>
    certificado_atributo = crear_certificado_atributo(clave_aa,
    certificado_digital, atributos)
  File "c:\Dlore python\python\simulacion_atributos.py", line 99, in
  crear_certificado_atributo
    x509.UnrecognizedExtension(
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~^
        oid=x509.ObjectIdentifier("1.2.3.4.5.6.7.8.1"),
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        value=atributos.encode(), # Codifica la cadena de atributos en
        bytes
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        ^^^^
        critical=False,
        ^^^^^^^^^^^^^^^
```